جامعة السلطان مولاي سليمان
Université Sultan Moulay Slimane
كلية العلوم التطبيقية - بني ملال
Faculté des Sciences Appliquées - Béni Mellal

OCP

PROJET DE FIN D'ÉTUDES

AI-Enhanced SOAR for Automated Incident Response in Enterprise SIEM

Une couche de classification par apprentissage automatique entre Wazuh et Shuffle

PRÉSENTÉ PAR

Hamza Saidi
Master Data Science et Sécurité des Systèmes d'Information

ENCADRANT PÉDAGOGIQUE

Pr. Mohamed El Fezazi
FSA Béni Mellal

ENCADRANTS PROFESSIONNELS

M. Fahmi Abderrahim
M. Erraji Hamza

ANNÉE UNIVERSITAIRE

2025 – 2026

CONTEXTE ET PROBLÈME

Un SOAR sans triage sature comme le SIEM

OCP Benguerir, laverie de Gantour · stage de 4 mois, mars à juillet 2026

≈100
alertes par seconde

SANS TRIAGE

SIEM — 100/s — SOAR — Analystes saturés

AVEC NOTRE COUCHE DE TRIAGE

SIEM — Classification ML — SOAR — Seul l'utile arrive

Idée proposée par l'auteur, **validée par l'encadrant OCP**

Objectifs et exigences

LE SYSTÈME DOIT

- **BF1** Surveiller les logs
- **BF2** Envoyer chaque alerte au SOAR
- **BF3** Classer LOW, MEDIUM, HIGH + score
- **BF4** Escalader selon le comportement
- **BF5** Bloquer l'IP des alertes HIGH
- **BF6** Notifier : Telegram et e-mail

CONTRAINTES

< 2 s
temps de réponse

10 Go
RAM, 4 vCPU

Docker
tous les composants

Natif
dataset depuis Wazuh

ÉTAT DE L'ART

Wazuh et Shuffle : les choix

Wazuh 4.9.2 — SIEM

- Open source
- Règles MITRE ATT&CK
- Webhook natif

Écartés : Splunk, QRadar, Elastic

Shuffle 2.2.0 — SOAR

- Open source et léger
- Blocs HTTP natifs
- Conditions sur les flèches

Écartés : TheHive, Cortex XSOAR, Swimlane

ÉTAT DE L'ART

Le ML pour la détection d’intrusions

Gu et al. (2019)

RF bat SVM et Naive Bayes

F1 98,7 % sur DoS

Sharafaldin et al. (2018)

Features réseau : **faible généralisation**

Catillo et al. (2022)

Domain shift sur les logs système

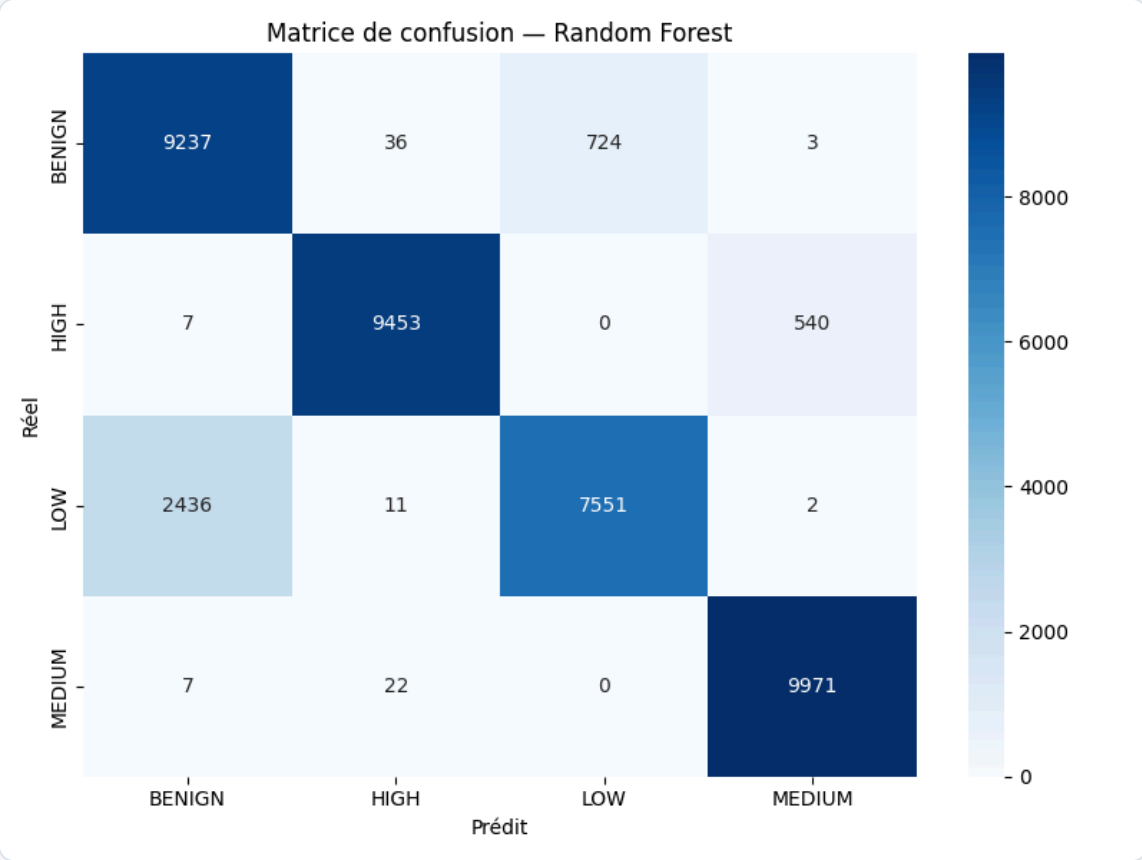
Hindy et al. (2020)

Features sémantiques **plus stables**

Notre approche : domain-native + comportement (fréquence, réputation d'IP)

ÉTAT DE L'ART

Pourquoi CICIDS2017 a échoué



Random Forest sur CICIDS2017

60,9 %

du pouvoir discriminant : features réseau **absentes des alertes Wazuh**

2 436

LOW prédits BENIGN

D'où un **dataset natif Wazuh**

CONCEPTION

Architecture en cinq couches

COUCHE 1

Simulation

simulate_attacks.py

4 types d'attaques

→

COUCHE 2

SIEM

Wazuh 4.9.2

Détecte et envoie en webhook

→

COUCHE 3

SOAR

Shuffle 2.2.0

Orchestre le flux

→

COUCHE 4

IA

Flask + Random Forest

Label et score

→

COUCHE 5

Réponse

Actions auto

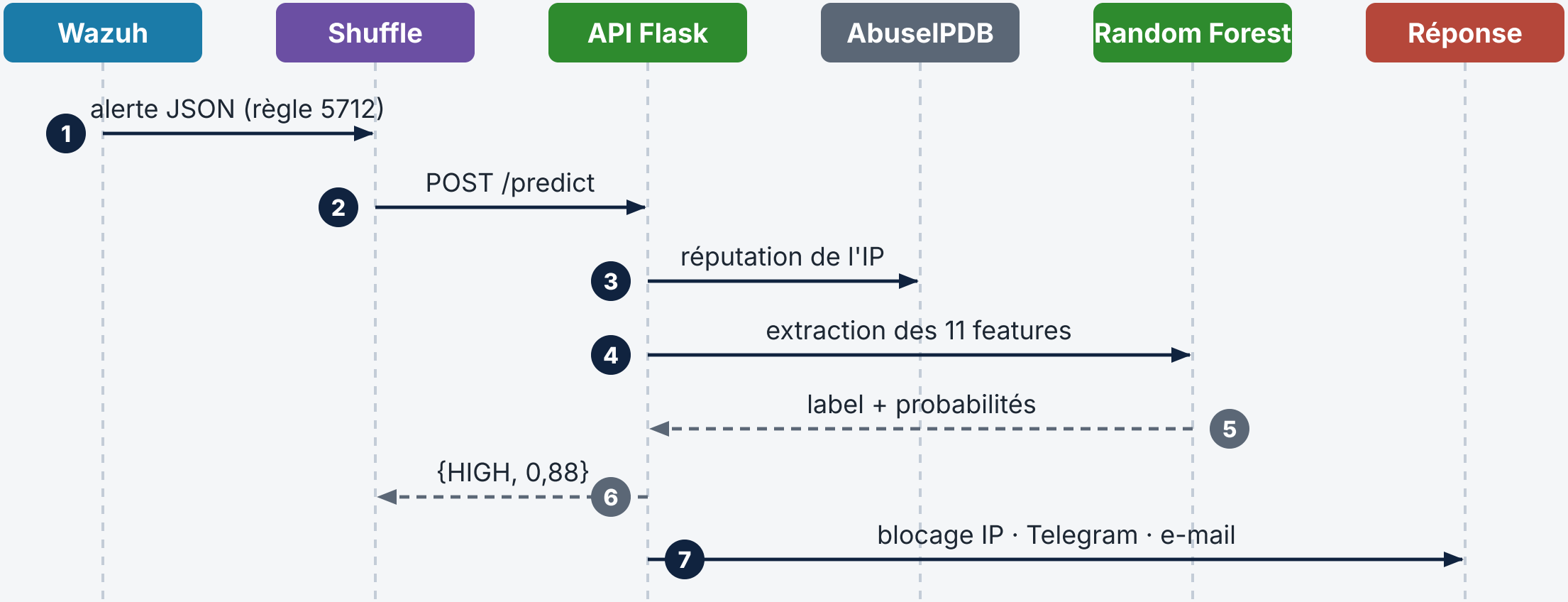
Blocage, Telegram, e-mail, journal

```
// réponse de POST /predict
{ "label": "HIGH",
  "confidence": 0.88,
  "action": "BLOCK_AND_ALERT" }
```

Couches reliées en **JSON** : chacune est remplaçable

CONCEPTION

Le parcours d'une alerte HIGH



Brute force SSH, règle Wazuh 5712 — **1,231 s** mesuré, objectif < 2 s

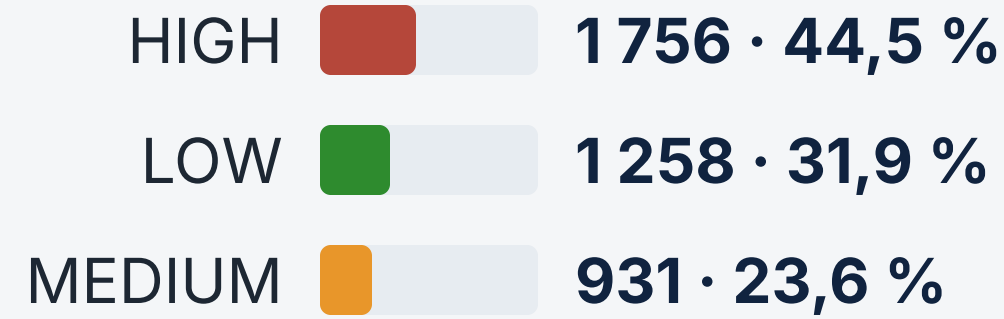
RÉALISATION

Le dataset : 3 945 alertes natives

1. Organiques
simulate_attacks.py

2. Injectées
niveaux 10 et 12

3. IP malveillantes
score AbuseIPDB



Découpage 80 / 20 stratifié : 789 alertes de test

RÉALISATION

Les 11 features

Natives Wazuh (6)

`rule_id_enc`	
`rule_level`	**26,85 %**
`groups_count`	
`has_mitre`	
`src_ip_private`	1,13 %
`fired_times`	5,17 %

Comportementales (4)

`alert_frequency_5min`	**13,10 %**
`repeat_rule_10min`	**13,84 %**
`hour_of_day`	
`is_business_hours`	1,78 %

Threat intel (1)

`ip_abuse_score`	**8,32 %**

AbuseIPDB, 0 si IP privée

RÉALISATION

Labellisation comportementale

Niveau Wazuh	Label	Escalade si...
7 et plus	HIGH	toujours
5 – 6	MEDIUM	fréq ≥ 10, rép ≥ 8 ou abuse ≥ 80 : HIGH
3 – 4	LOW	fréq ≥ 15, rép ≥ 10 ou abuse ≥ 80 : MEDIUM

Même niveau, labels différents

Labels corrélés aux features

voir limites

RÉALISATION

Random Forest et API Flask

Déséquilibre géré
class_weight balanced, 200 arbres

Interprétable
feature_importances_

Portable
CPU et GPU

HTTP	Endpoint
POST	`/predict`
POST	`/respond/high`
POST	`/respond/medium`
POST	`/respond/low`
GET	`/health` · `/stats`

Un seul appel depuis Shuffle : `/predict` déclenche la bonne réponse

RÉALISATION

Workflow Shuffle et actions

Webhook → POST /predict → Extraction du label → 3 branches

HIGH

Blocage IP

Telegram enrichi, e-mail

MEDIUM

Notification

Telegram compact, e-mail

LOW

Journal

incidents.json

RÉSULTATS

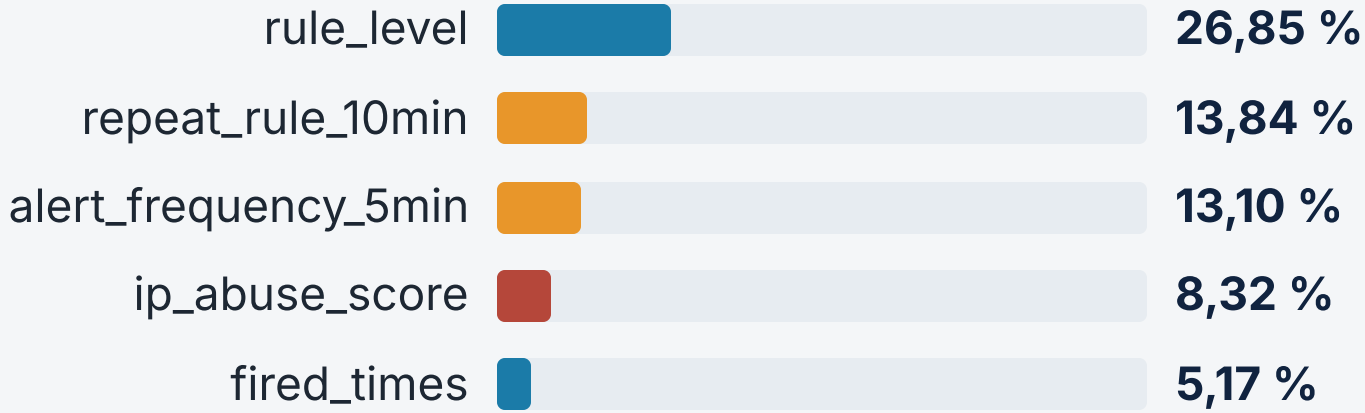
Résultats du modèle V2

100 %

accuracy, 789 alertes de test

Classe	F1	Support
HIGH	1,00	352
LOW	1,00	302
MEDIUM	1,00	135

IMPORTANCE DES FEATURES



`rule_level` : **39,74 % (V1) à 26,85 % (V2)**

RÉSULTAT CLÉ

Escalade comportementale

Même alerte, `rule.level = 5`, répétée 9 fois depuis la même source

	1	2	3	4	5	6	7	8	9
Répétitions sur 10 min repeat_rule_10min	0	1	2	3	4	5	6	7	8
Seuil Shuffle rule.level ≥ 7	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM
Notre Random Forest	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	MEDIUM	HIGH

9e alerte : **HIGH (58 %)**. Un seuil sur le niveau ne le peut pas.

IP publique, AbuseIPDB 100 : **HIGH** malgré un niveau faible.

VALIDATION

Tests de bout en bout

LOW

98 %

Règle 5715

Journal

MEDIUM

63 %

Règle 31103 (XSS)

Notification

HIGH

94 %

Règle 5712 (SSH)

Blocage + alertes

Réponse HIGH

`blocked` `telegram_sent` `email_sent` : **true**

1,231 s

objectif < 2 s

BILAN

Mes contributions

Problème et idée

Triage avant le SOAR

Diagnostic CICIDS

Changement d'approche

Features

Temporelles et AbuseIPDB

Labellisation

Escalade comportementale

API Flask V2

6 endpoints, Active Response

Infrastructure

Docker, LVM, Orborus

DISCUSSION

Limites et perspectives

Limites

- 100 % = cohérence interne
- Dataset de laboratoire
- IP simulées (score = 0)
- Blocage : agent Wazuh requis
- API en mode développement

Perspectives

- Données de production OCP
- Agents Wazuh sur les serveurs
- Docker + Gunicorn
- LSTM sur séquences d'alertes
- Mesures de latence (50+)

CONCLUSION

Ce qu’il faut retenir

Problème

100 alertes/s, SOAR saturé

Solution

Random Forest entre Wazuh et Shuffle

Résultat

Pipeline complet en 1,231 s

Apport central : même niveau Wazuh, criticités différentes selon le comportement



Merci de votre attention

Place à vos questions